

```
# Not so good
select_input()
checkbox_input()
text_input()
```

A good example of this design is the **stringr** package: if you don't remember exactly which function you need, you can type `str_` and jog your memory.

Where possible, avoid overriding existing functions and variables. It's impossible to do in general because so many good names are already taken by other packages, but avoiding the most common names from base R will avoid confusion:

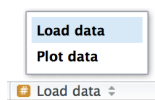
```
# Don't do this!
T <- FALSE
c <- 10
mean <- function(x) sum(x)
```

Use comments, lines starting with `#`, to explain the “why” of your code. You generally should avoid comments that explain the “what” or the “how.” If you can't understand what the code does from reading it, you should think about how to rewrite it to be more clear. Do you need to add some intermediate variables with useful names? Do you need to break out a subcomponent of a large function so you can name it? However, your code can never capture the reasoning behind your decisions: why did you choose this approach instead of an alternative? What else did you try that didn't work? It's a great idea to capture that sort of thinking in a comment.

Another important use of comments is to break up your file into easily readable chunks. Use long lines of `-` or `=` to make it easy to spot the breaks:

```
# Load data -----
# Plot data -----
```

RStudio provides a keyboard shortcut to create these headers (`Cmd/Ctrl-Shift-R`), and will display them in the code navigation drop-down at the bottom-left of the editor:



Exercises

1. Read the source code for each of the following three functions, puzzle out what they do, and then brainstorm better names:

```
f1 <- function(string, prefix) {  
  substr(string, 1, nchar(prefix)) == prefix  
}  
f2 <- function(x) {  
  if (length(x) <= 1) return(NULL)  
  x[-length(x)]  
}  
f3 <- function(x, y) {  
  rep(y, length.out = length(x))  
}
```

2. Take a function that you've written recently and spend five minutes brainstorming a better name for it and its arguments.
3. Compare and contrast `rnorm()` and `MASS::mvrnorm()`. How could you make them more consistent?
4. Make a case for why `norm_r()`, `norm_d()`, etc., would be better than `rnorm()`, `dnorm()`. Make a case for the opposite.

Conditional Execution

An `if` statement allows you to conditionally execute code. It looks like this:

```
if (condition) {  
  # code executed when condition is TRUE  
} else {  
  # code executed when condition is FALSE  
}
```

To get help on `if` you need to surround it in backticks: `?`if``. The help isn't particularly helpful if you're not already an experienced programmer, but at least you know how to get to it!

Here's a simple function that uses an `if` statement. The goal of this function is to return a logical vector describing whether or not each element of a vector is named:

```
has_name <- function(x) {  
  nms <- names(x)  
  if (is.null(nms)) {  
    rep(FALSE, length(x))  
  }
```

```

    } else {
      !is.na(nms) & nms != ""
    }
  }
}

```

This function takes advantage of the standard return rule: a function returns the last value that it computed. Here that is either one of the two branches of the `if` statement.

Conditions

The condition must evaluate to either `TRUE` or `FALSE`. If it's a vector, you'll get a warning message; if it's an `NA`, you'll get an error. Watch out for these messages in your own code:

```

if (c(TRUE, FALSE)) {}
#> Warning in if (c(TRUE, FALSE)) {:
#> the condition has length > 1 and only the
#> first element will be used
#> NULL

if (NA) {}
#> Error in if (NA) {: missing value where TRUE/FALSE needed

```

You can use `||` (or) and `&&` (and) to combine multiple logical expressions. These operators are “short-circuiting”: as soon as `||` sees the first `TRUE` it returns `TRUE` without computing anything else. As soon as `&&` sees the first `FALSE` it returns `FALSE`. You should never use `|` or `&` in an `if` statement: these are vectorized operations that apply to multiple values (that's why you use them in `filter()`). If you do have a logical vector, you can use `any()` or `all()` to collapse it to a single value.

Be careful when testing for equality. `==` is vectorized, which means that it's easy to get more than one output. Either check the length is already 1, collapse with `all()` or `any()`, or use the nonvectorized `identical()`. `identical()` is very strict: it always returns either a single `TRUE` or a single `FALSE`, and doesn't coerce types. This means that you need to be careful when comparing integers and doubles:

```

identical(0L, 0)
#> [1] FALSE

```

You also need to be wary of floating-point numbers:

```

x <- sqrt(2) ^ 2
x
#> [1] 2

```

```
x == 2
#> [1] FALSE
x - 2
#> [1] 4.44e-16
```

Instead use `dplyr::near()` for comparisons, as described in “[Comparisons](#)” on page 46.

And remember, `x == NA` doesn’t do anything useful!

Multiple Conditions

You can chain multiple `if` statements together:

```
if (this) {
  # do that
} else if (that) {
  # do something else
} else {
  #
}
```

But if you end up with a very long series of chained `if` statements, you should consider rewriting. One useful technique is the `switch()` function. It allows you to evaluate selected code based on position or name:

```
#> function(x, y, op) {
#>   switch(op,
#>     plus = x + y,
#>     minus = x - y,
#>     times = x * y,
#>     divide = x / y,
#>     stop("Unknown op!")
#>   )
#> }
```

Another useful function that can often eliminate long chains of `if` statements is `cut()`. It’s used to discretize continuous variables.

Code Style

Both `if` and `function` should (almost) always be followed by squiggly brackets (`{}`), and the contents should be indented by two spaces. This makes it easier to see the hierarchy in your code by skimming the lefthand margin.

An opening curly brace should never go on its own line and should always be followed by a new line. A closing curly brace should

always go on its own line, unless it's followed by `else`. Always indent the code inside curly braces:

```
# Good
if (y < 0 && debug) {
  message("Y is negative")
}

if (y == 0) {
  log(x)
} else {
  y ^ x
}

# Bad
if (y < 0 && debug)
  message("Y is negative")

if (y == 0) {
  log(x)
}
else {
  y ^ x
}
```

It's OK to drop the curly braces if you have a very short `if` statement that can fit on one line:

```
y <- 10
x <- if (y < 20) "Too low" else "Too high"
```

I recommend this only for very brief `if` statements. Otherwise, the full form is easier to read:

```
if (y < 20) {
  x <- "Too low"
} else {
  x <- "Too high"
}
```

Exercises

1. What's the difference between `if` and `ifelse()`? Carefully read the help and construct three examples that illustrate the key differences.
2. Write a greeting function that says “good morning,” “good afternoon,” or “good evening,” depending on the time of day. (Hint: use a time argument that defaults to `lubridate::now()`. That will make it easier to test your function.)

3. Implement a `fizzbuzz` function. It takes a single number as input. If the number is divisible by three, it returns “fizz”. If it’s divisible by five it returns “buzz”. If it’s divisible by three and five, it returns “fizzbuzz”. Otherwise, it returns the number. Make sure you first write working code before you create the function.
4. How could you use `cut()` to simplify this set of nested if-else statements?

```
if (temp <= 0) {  
  "freezing"  
} else if (temp <= 10) {  
  "cold"  
} else if (temp <= 20) {  
  "cool"  
} else if (temp <= 30) {  
  "warm"  
} else {  
  "hot"  
}
```

How would you change the call to `cut()` if I’d used `<` instead of `<=`? What is the other chief advantage of `cut()` for this problem? (Hint: what happens if you have many values in `temp`?)

5. What happens if you use `switch()` with numeric values?
6. What does this `switch()` call do? What happens if `x` is “e”?

```
switch(x,  
  a = ,  
  b = "ab",  
  c = ,  
  d = "cd"  
)
```

Experiment, then carefully read the documentation.

Function Arguments

The arguments to a function typically fall into two broad sets: one set supplies the *data* to compute on, and the other supplies arguments that control the *details* of the computation. For example:

- In `log()`, the data is `x`, and the detail is the base of the logarithm.

- In `mean()`, the data is `x`, and the details are how much data to trim from the ends (`trim`) and how to handle missing values (`na.rm`).
- In `t.test()`, the data are `x` and `y`, and the details of the test are `alternative`, `mu`, `paired`, `var.equal`, and `conf.level`.
- In `str_c()` you can supply any number of strings to `...`, and the details of the concatenation are controlled by `sep` and `collapse`.

Generally, data arguments should come first. Detail arguments should go on the end, and usually should have default values. You specify a default value in the same way you call a function with a named argument:

```
# Compute confidence interval around
# mean using normal approximation
mean_ci <- function(x, conf = 0.95) {
  se <- sd(x) / sqrt(length(x))
  alpha <- 1 - conf
  mean(x) + se * qnorm(c(alpha / 2, 1 - alpha / 2))
}

x <- runif(100)
mean_ci(x)
#> [1] 0.498 0.610
mean_ci(x, conf = 0.99)
#> [1] 0.480 0.628
```

The default value should almost always be the most common value. The few exceptions to this rule have to do with safety. For example, it makes sense for `na.rm` to default to `FALSE` because missing values are important. Even though `na.rm = TRUE` is what you usually put in your code, it's a bad idea to silently ignore missing values by default.

When you call a function, you typically omit the names of the data arguments, because they are used so commonly. If you override the default value of a detail argument, you should use the full name:

```
# Good
mean(1:10, na.rm = TRUE)

# Bad
mean(x = 1:10, , FALSE)
mean(, TRUE, x = c(1:10, NA))
```

You can refer to an argument by its unique prefix (e.g., `mean(x, n = TRUE)`), but this is generally best avoided given the possibilities for confusion.

Notice that when you call a function, you should place a space around `=` in function calls, and always put a space after a comma, not before (just like in regular English). Using whitespace makes it easier to skim the function for the important components:

```
# Good
average <- mean(feet / 12 + inches, na.rm = TRUE)

# Bad
average<-mean(feet/12+inches,na.rm=TRUE)
```

Choosing Names

The names of the arguments are also important. R doesn't care, but the readers of your code (including future-you!) will. Generally you should prefer longer, more descriptive names, but there are a handful of very common, very short names. It's worth memorizing these:

- `x`, `y`, `z`: vectors.
- `w`: a vector of weights.
- `df`: a data frame.
- `i`, `j`: numeric indices (typically rows and columns).
- `n`: length, or number of rows.
- `p`: number of columns.

Otherwise, consider matching names of arguments in existing R functions. For example, use `na.rm` to determine if missing values should be removed.

Checking Values

As you start to write more functions, you'll eventually get to the point where you don't remember exactly how your function works. At this point it's easy to call your function with invalid inputs. To avoid this problem, it's often useful to make constraints explicit. For example, imagine you've written some functions for computing weighted summary statistics:


```
wt_mean <- function(x, w) {
  sum(x * w) / sum(x)
}
wt_var <- function(x, w) {
  mu <- wt_mean(x, w)
  sum(w * (x - mu) ^ 2) / sum(w)
}
wt_sd <- function(x, w) {
  sqrt(wt_var(x, w))
}
```

What happens if `x` and `w` are not the same length?

```
wt_mean(1:6, 1:3)
#> [1] 2.19
```

In this case, because of R's vector recycling rules, we don't get an error.

It's good practice to check important preconditions, and throw an error (with `stop()`) if they are not true:

```
wt_mean <- function(x, w) {
  if (length(x) != length(w)) {
    stop("`x` and `w` must be the same length", call. = FALSE)
  }
  sum(w * x) / sum(x)
}
```

Be careful not to take this too far. There's a trade-off between how much time you spend making your function robust, versus how long you spend writing it. For example, if you also added a `na.rm` argument, I probably wouldn't check it carefully:

```
wt_mean <- function(x, w, na.rm = FALSE) {
  if (!is.logical(na.rm)) {
    stop("`na.rm` must be logical")
  }
  if (length(na.rm) != 1) {
    stop("`na.rm` must be length 1")
  }
  if (length(x) != length(w)) {
    stop("`x` and `w` must be the same length", call. = FALSE)
  }

  if (na.rm) {
    miss <- is.na(x) | is.na(w)
    x <- x[!miss]
    w <- w[!miss]
  }
  sum(w * x) / sum(x)
}
```

This is a lot of extra work for little additional gain. A useful compromise is the built-in `stopifnot()`; it checks that each argument is `TRUE`, and produces a generic error message if not:

```
wt_mean <- function(x, w, na.rm = FALSE) {
  stopifnot(is.logical(na.rm), length(na.rm) == 1)
  stopifnot(length(x) == length(w))

  if (na.rm) {
    miss <- is.na(x) | is.na(w)
    x <- x[!miss]
    w <- w[!miss]
  }
  sum(w * x) / sum(x)
}
wt_mean(1:6, 6:1, na.rm = "foo")
#> Error: is.logical(na.rm) is not TRUE
```

Note that when using `stopifnot()` you assert what should be true rather than checking for what might be wrong.

Dot-Dot-Dot (...)

Many functions in R take an arbitrary number of inputs:

```
sum(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
#> [1] 55
stringr::str_c("a", "b", "c", "d", "e", "f")
#> [1] "abcdef"
```

How do these functions work? They rely on a special argument: `...` (pronounced dot-dot-dot). This special argument captures any number of arguments that aren't otherwise matched.

It's useful because you can then send those `...` on to another function. This is a useful catch-all if your function primarily wraps another function. For example, I commonly create these helper functions that wrap around `str_c()`:

```
commas <- function(...) stringr::str_c(..., collapse = ", ")
commas(letters[1:10])
#> [1] "a, b, c, d, e, f, g, h, i, j"

rule <- function(..., pad = "-") {
  title <- paste0(...)
  width <- getOption("width") - nchar(title) - 5
  cat(title, " ", stringr::str_dup(pad, width), "\n", sep = "")
}
rule("Important output")
#> Important output -----
```

Here ... lets me forward on any arguments that I don't want to deal with to `str_c()`. It's a very convenient technique. But it does come at a price: any misspelled arguments will not raise an error. This makes it easy for typos to go unnoticed:

```
x <- c(1, 2)
sum(x, na.rm = TRUE)
#> [1] 4
```

If you just want to capture the values of the ..., use `list(...)`.

Lazy Evaluation

Arguments in R are lazily evaluated: they're not computed until they're needed. That means if they're never used, they're never called. This is an important property of R as a programming language, but is generally not important when you're writing your own functions for data analysis. You can read more about lazy evaluation at <http://adv-r.had.co.nz/Functions.html#lazy-evaluation>.

Exercises

1. What does `commas(letters, collapse = "-")` do? Why?
2. It'd be nice if you could supply multiple characters to the `pad` argument, e.g., `rule("Title", pad = "-+")`. Why doesn't this currently work? How could you fix it?
3. What does the `trim` argument to `mean()` do? When might you use it?
4. The default value for the `method` argument to `cor()` is `c("pearson", "kendall", "spearman")`. What does that mean? What value is used by default?

Return Values

Figuring out what your function should return is usually straightforward: it's why you created the function in the first place! There are two things you should consider when returning a value:

- Does returning early make your function easier to read?
- Can you make your function pipeable?

Explicit Return Statements

The value returned by the function is usually the last statement it evaluates, but you can choose to return early by using `return()`. I think it's best to save the use of `return()` to signal that you can return early with a simpler solution. A common reason to do this is because the inputs are empty:

```
complicated_function <- function(x, y, z) {  
  if (length(x) == 0 || length(y) == 0) {  
    return(0)  
  }  
  
  # Complicated code here  
}
```

Another reason is because you have a `if` statement with one complex block and one simple block. For example, you might write an `if` statement like this:

```
f <- function() {  
  if (x) {  
    # Do  
    # something  
    # that  
    # takes  
    # many  
    # lines  
    # to  
    # express  
  } else {  
    # return something short  
  }  
}
```

But if the first block is very long, by the time you get to the `else`, you've forgotten the condition. One way to rewrite it is to use an early return for the simple case:

```
f <- function() {  
  if (!x) {  
    return(something_short)  
  }  
  
  # Do  
  # something  
  # that  
  # takes  
  # many  
  # lines
```

```

    # to
    # express
  }

```

This tends to make the code easier to understand, because you don't need quite so much context to understand it.

Writing Pipeable Functions

If you want to write your own pipeable functions, thinking about the return value is important. There are two main types of pipeable functions: transformation and side-effect.

In *transformation* functions, there's a clear "primary" object that is passed in as the first argument, and a modified version is returned by the function. For example, the key objects for **dplyr** and **tidyr** are data frames. If you can identify what the object type is for your domain, you'll find that your functions just work with the pipe.

Side-effect functions are primarily called to perform an action, like drawing a plot or saving a file, not transforming an object. These functions should "invisibly" return the first argument, so they're not printed by default, but can still be used in a pipeline. For example, this simple function prints out the number of missing values in a data frame:

```

show_missings <- function(df) {
  n <- sum(is.na(df))
  cat("Missing values: ", n, "\n", sep = "")
  invisible(df)
}

```

If we call it interactively, the `invisible()` means that the input `df` doesn't get printed out:

```

show_missings(mtcars)
#> Missing values: 0

```

But it's still there, it's just not printed by default:

```

x <- show_missings(mtcars)
#> Missing values: 0
class(x)
#> [1] "data.frame"
dim(x)
#> [1] 32 11

```

And we can still use it in a pipe:

```
mtcars %>%  
  show_missings() %>%  
  mutate(mpg = ifelse(mpg < 20, NA, mpg)) %>%  
  show_missings()  
#> Missing values: 0  
#> Missing values: 18
```

Environment

The last component of a function is its environment. This is not something you need to understand deeply when you first start writing functions. However, it's important to know a little bit about environments because they are crucial to how functions work. The environment of a function controls how R finds the value associated with a name. For example, take this function:

```
f <- function(x) {  
  x + y  
}
```

In many programming languages, this would be an error, because *y* is not defined inside the function. In R, this is valid code because R uses rules called *lexical scoping* to find the value associated with a name. Since *y* is not defined inside the function, R will look in the *environment* where the function was defined:

```
y <- 100  
f(10)  
#> [1] 110  
  
y <- 1000  
f(10)  
#> [1] 1010
```

This behavior seems like a recipe for bugs, and indeed you should avoid creating functions like this deliberately, but by and large it doesn't cause too many problems (especially if you regularly restart R to get to a clean slate).

The advantage of this behavior is that from a language standpoint it allows R to be very consistent. Every name is looked up using the same set of rules. For `f()` that includes the behavior of two things that you might not expect: `{` and `+`. This allows you to do devious things like:

```

`+` <- function(x, y) {
  if (runif(1) < 0.1) {
    sum(x, y)
  } else {
    sum(x, y) * 1.1
  }
}
table(replicate(1000, 1 + 2))
#>
#>   3 3.3
#> 100 900
rm(`+`)

```

This is a common phenomenon in R. R places few limits on your power. You can do many things that you can't do in other programming languages. You can do many things that 99% of the time are extremely ill-advised (like overriding how addition works!). But this power and flexibility is what makes tools like **ggplot2** and **dplyr** possible. Learning how to make best use of this flexibility is beyond the scope of this book, but you can read about in *Advanced R*.

Introduction

So far this book has focused on tibbles and packages that work with them. But as you start to write your own functions, and dig deeper into R, you need to learn about vectors, the objects that underlie tibbles. If you've learned R in a more traditional way, you're probably already familiar with vectors, as most R resources start with vectors and work their way up to tibbles. I think it's better to start with tibbles because they're immediately useful, and then work your way down to the underlying components.

Vectors are particularly important as most of the functions you will write will work with vectors. It is possible to write functions that work with tibbles (like in **ggplot2**, **dplyr**, and **tidyr**), but the tools you need to write such functions are currently idiosyncratic and immature. I am working on a better approach, <https://github.com/hadley/lazyeval>, but it will not be ready in time for the publication of the book. Even when complete, you'll still need to understand vectors; it'll just make it easier to write a user-friendly layer on top.

Prerequisites

The focus of this chapter is on base R data structures, so it isn't essential to load any packages. We will, however, use a handful of functions from the **purrr** package to avoid some inconsistencies in base R.

```
library(tidyverse)
#> Loading tidyverse: ggplot2
#> Loading tidyverse: tibble
#> Loading tidyverse: tidyr
#> Loading tidyverse: readr
#> Loading tidyverse: purrr
#> Loading tidyverse: dplyr
#> Conflicts with tidy packages -----
#> filter(): dplyr, stats
#> lag():    dplyr, stats
```

Vector Basics

There are two types of vectors:

- *Atomic* vectors, of which there are six types: *logical*, *integer*, *double*, *character*, *complex*, and *raw*. Integer and double vectors are collectively known as *numeric* vectors.
- *Lists*, which are sometimes called recursive vectors because lists can contain other lists.

The chief difference between atomic vectors and lists is that atomic vectors are *homogeneous*, while lists can be *heterogeneous*. There's one other related object: `NULL`. `NULL` is often used to represent the absence of a vector (as opposed to `NA`, which is used to represent the absence of a value in a vector). `NULL` typically behaves like a vector of length 0. [Figure 16-1](#) summarizes the interrelationships.

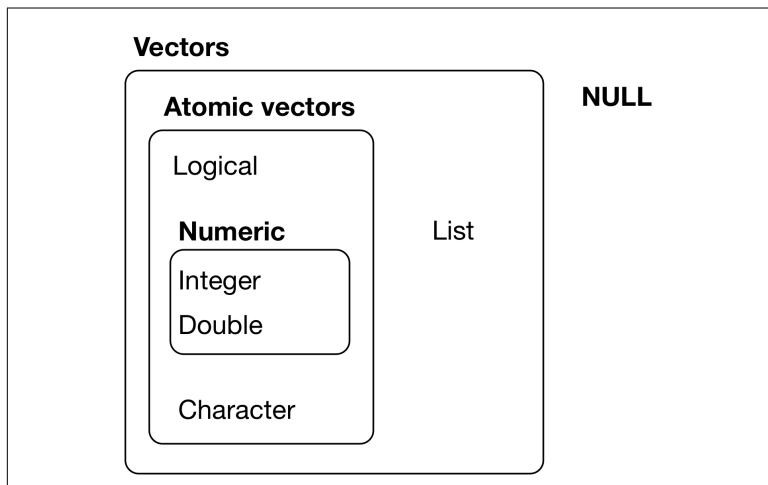


Figure 16-1. The hierarchy of R's vector types

Every vector has two key properties:

- Its *type*, which you can determine with `typeof()`:

```
typeof(letters)
#> [1] "character"
typeof(1:10)
#> [1] "integer"
```

- Its *length*, which you can determine with `length()`:

```
x <- list("a", "b", 1:10)
length(x)
#> [1] 3
```

Vectors can also contain arbitrary additional metadata in the form of attributes. These attributes are used to create *augmented vectors*, which build on additional behavior. There are four important types of augmented vector:

- Factors are built on top of integer vectors.
- Dates and date-times are built on top of numeric vectors.
- Data frames and tibbles are built on top of lists.

This chapter will introduce you to these important vectors from simplest to most complicated. You'll start with atomic vectors, then build up to lists, and finish off with augmented vectors.

Important Types of Atomic Vector

The four most important types of atomic vector are logical, integer, double, and character. Raw and complex are rarely used during a data analysis, so I won't discuss them here.

Logical

Logical vectors are the simplest type of atomic vector because they can take only three possible values: FALSE, TRUE, and NA. Logical vectors are usually constructed with comparison operators, as described in [“Comparisons” on page 46](#). You can also create them by hand with `c()`:

```
1:10 %% 3 == 0
#> [1] FALSE FALSE TRUE FALSE FALSE
#> [2] TRUE FALSE FALSE TRUE FALSE
```

```
c(TRUE, TRUE, FALSE, NA)
#> [1] TRUE TRUE FALSE NA
```

Numeric

Integer and double vectors are known collectively as numeric vectors. In R, numbers are doubles by default. To make an integer, place a `L` after the number:

```
typeof(1)
#> [1] "double"
typeof(1L)
#> [1] "integer"
1.5L
#> [1] 1.5
```

The distinction between integers and doubles is not usually important, but there are two important differences that you should be aware of:

- Doubles are approximations. Doubles represent floating-point numbers that cannot always be precisely represented with a fixed amount of memory. This means that you should consider all doubles to be approximations. For example, what is square of the square root of two?

```
x <- sqrt(2) ^ 2
x
#> [1] 2
x - 2
#> [1] 4.44e-16
```

This behavior is common when working with floating-point numbers: most calculations include some approximation error. Instead of comparing floating-point numbers using `==`, you should use `dplyr::near()`, which allows for some numerical tolerance.

- Integers have one special value, `NA`, while doubles have four, `NA`, `NaN`, `Inf`, and `-Inf`. All three special values can arise during division:

```
c(-1, 0, 1) / 0
#> [1] -Inf NaN Inf
```

Avoid using `==` to check for these other special values. Instead use the helper functions `is.finite()`, `is.infinite()`, and `is.nan()`:

	0	Inf	NA	NaN
<code>is.finite()</code>	x			
<code>is.infinite()</code>		x		
<code>is.na()</code>			x	x
<code>is.nan()</code>				x

Character

Character vectors are the most complex type of atomic vector, because each element of a character vector is a string, and a string can contain an arbitrary amount of data.

You've already learned a lot about working with strings in [Chapter 11](#). Here I want to mention one important feature of the underlying string implementation: R uses a global string pool. This means that each unique string is only stored in memory once, and every use of the string points to that representation. This reduces the amount of memory needed by duplicated strings. You can see this behavior in practice with `pryr::object_size()`:

```
x <- "This is a reasonably long string."
pryr::object_size(x)
#> 136 B

y <- rep(x, 1000)
pryr::object_size(y)
#> 8.13 kB
```

`y` doesn't take up 1000x as much memory as `x`, because each element of `y` is just a pointer to that same string. A pointer is 8 bytes, so 1000 pointers to a 136 B string is $8 * 1000 + 136 = 8.13$ kB.

Missing Values

Note that each type of atomic vector has its own missing value:

```
NA           # logical
#> [1] NA
NA_integer_  # integer
#> [1] NA
NA_real_     # double
#> [1] NA
NA_character_ # character
#> [1] NA
```

Normally you don't need to know about these different types because you can always use `NA` and it will be converted to the correct type using the implicit coercion rules described next. However, there are some functions that are strict about their inputs, so it's useful to have this knowledge sitting in your back pocket so you can be specific when needed.

Exercises

1. Describe the difference between `is.finite(x)` and `!is.infinite(x)`.
2. Read the source code for `dplyr::near()` (Hint: to see the source code, drop the `()`). How does it work?
3. A logical vector can take three possible values. How many possible values can an integer vector take? How many possible values can a double take? Use Google to do some research.
4. Brainstorm at least four functions that allow you to convert a double to an integer. How do they differ? Be precise.
5. What functions from the **readr** package allow you to turn a string into a logical, integer, and double vector?

Using Atomic Vectors

Now that you understand the different types of atomic vector, it's useful to review some of the important tools for working with them. These include:

- How to convert from one type to another, and when that happens automatically.
- How to tell if an object is a specific type of vector.
- What happens when you work with vectors of different lengths.
- How to name the elements of a vector.
- How to pull out elements of interest.

Coercion

There are two ways to convert, or coerce, one type of vector to another:

- Explicit coercion happens when you call a function like `as.logical()`, `as.integer()`, `as.double()`, or `as.character()`. Whenever you find yourself using explicit coercion, you should always check whether you can make the fix upstream, so that the vector never had the wrong type in the first place. For example, you may need to tweak your **readr** `col_types` specification.
- Implicit coercion happens when you use a vector in a specific context that expects a certain type of vector. For example, when you use a logical vector with a numeric summary function, or when you use a double vector where an integer vector is expected.

Because explicit coercion is used relatively rarely, and is largely easy to understand, I'll focus on implicit coercion here.

You've already seen the most important type of implicit coercion: using a logical vector in a numeric context. In this case `TRUE` is converted to 1 and `FALSE` is converted to 0. That means the sum of a logical vector is the number of trues, and the mean of a logical vector is the proportion of trues:

```
x <- sample(20, 100, replace = TRUE)
y <- x > 10
sum(y) # how many are greater than 10?
#> [1] 44
mean(y) # what proportion are greater than 10?
#> [1] 0.44
```

You may see some code (typically older) that relies on implicit coercion in the opposite direction, from integer to logical:

```
if (length(x)) {
  # do something
}
```

In this case, 0 is converted to `FALSE` and everything else is converted to `TRUE`. I think this makes it harder to understand your code, and I don't recommend it. Instead be explicit: `length(x) > 0`.

It's also important to understand what happens when you try and create a vector containing multiple types with `c()`—the most complex type always wins:

```
typeof(c(TRUE, 1L))
#> [1] "integer"
typeof(c(1L, 1.5))
```

```
#> [1] "double"
typeof(c(1.5, "a"))
#> [1] "character"
```

An atomic vector cannot have a mix of different types because the type is a property of the complete vector, not the individual elements. If you need to mix multiple types in the same vector, you should use a list, which you’ll learn about shortly.

Test Functions

Sometimes you want to do different things based on the type of vector. One option is to use `typeof()`. Another is to use a test function that returns a TRUE or FALSE. Base R provides many functions like `is.vector()` and `is.atomic()`, but they often return surprising results. Instead, it’s safer to use the `is_*` functions provided by **purrr**, which are summarized in the following table.

	lgl	int	dbl	chr	list
<code>is_logical()</code>	x				
<code>is_integer()</code>		x			
<code>is_double()</code>			x		
<code>is_numeric()</code>		x	x		
<code>is_character()</code>				x	
<code>is_atomic()</code>	x	x	x	x	
<code>is_list()</code>					x
<code>is_vector()</code>	x	x	x	x	x

Each predicate also comes with a “scalar” version, like `is_scalar_atomic()`, which checks that the length is 1. This is useful, for example, if you want to check that an argument to your function is a single logical value.

Scalars and Recycling Rules

As well as implicitly coercing the types of vectors to be compatible, R will also implicitly coerce the length of vectors. This is called vector *recycling*, because the shorter vector is repeated, or recycled, to the same length as the longer vector.

This is generally most useful when you are mixing vectors and “scalars.” I put scalars in quotes because R doesn’t actually have

scalars: instead, a single number is a vector of length 1. Because there are no scalars, most built-in functions are *vectorized*, meaning that they will operate on a vector of numbers. That's why, for example, this code works:

```
sample(10) + 100
#> [1] 109 108 104 102 103 110 106 107 105 101
runif(10) > 0.5
#> [1] TRUE TRUE FALSE TRUE TRUE TRUE FALSE TRUE TRUE
#> [10] TRUE
```

In R, basic mathematical operations work with vectors. That means that you should never need to perform explicit iteration when performing simple mathematical computations.

It's intuitive what should happen if you add two vectors of the same length, or a vector and a “scalar,” but what happens if you add two vectors of different lengths?

```
1:10 + 1:2
#> [1] 2 4 4 6 6 8 8 10 10 12
```

Here, R will expand the shortest vector to the same length as the longest, so-called recycling. This is silent except when the length of the longer is not an integer multiple of the length of the shorter:

```
1:10 + 1:3
#> Warning in 1:10 + 1:3:
#> longer object length is not a multiple of shorter
#> object length
#> [1] 2 4 6 5 7 9 8 10 12 11
```

While vector recycling can be used to create very succinct, clever code, it can also silently conceal problems. For this reason, the vectorized functions in tidyverse will throw errors when you recycle anything other than a scalar. If you do want to recycle, you'll need to do it yourself with `rep()`:

```
tibble(x = 1:4, y = 1:2)
#> Error: Variables must be length 1 or 4.
#> Problem variables: 'y'

tibble(x = 1:4, y = rep(1:2, 2))
#> # A tibble: 4 × 2
#>       x     y
#>   <int> <int>
#> 1     1     1
#> 2     2     2
#> 3     3     1
#> 4     4     2
```

```
tibble(x = 1:4, y = rep(1:2, each = 2))
#> # A tibble: 4 × 2
#>       x     y
#>   <int> <int>
#> 1     1     1
#> 2     2     1
#> 3     3     2
#> 4     4     2
```

Naming Vectors

All types of vectors can be named. You can name them during creation with `c()`:

```
c(x = 1, y = 2, z = 4)
#> x y z
#> 1 2 4
```

Or after the fact with `purrr::set_names()`:

```
set_names(1:3, c("a", "b", "c"))
#> a b c
#> 1 2 3
```

Named vectors are most useful for subsetting, described next.

Subsetting

So far we've used `dplyr::filter()` to filter the rows in a tibble. `filter()` only works with tibble, so we'll need a new tool for vectors: `[[`. `[[` is the subsetting function, and is called like `x[a]`. There are four types of things that you can subset a vector with:

- A numeric vector containing only integers. The integers must either be all positive, all negative, or zero.

Subsetting with positive integers keeps the elements at those positions:

```
x <- c("one", "two", "three", "four", "five")
x[[3, 2, 5]]
#> [1] "three" "two" "five"
```

By repeating a position, you can actually make a longer output than input:

```
x[[c(1, 1, 5, 5, 5, 2)]]
#> [1] "one" "one" "five" "five" "five" "two"
```

Negative values drop the elements at the specified positions:

```
x[c(-1, -3, -5)]  
#> [1] "two" "four"
```

It's an error to mix positive and negative values:

```
x[c(1, -1)]  
#> Error in x[c(1, -1)]:  
#> only 0's may be mixed with negative subscripts
```

The error message mentions subsetting with zero, which returns no values:

```
x[0]  
#> character(0)
```

This is not useful very often, but it can be helpful if you want to create unusual data structures to test your functions with.

- Subsetting with a logical vector keeps all values corresponding to a TRUE value. This is most often useful in conjunction with the comparison functions:

```
x <- c(10, 3, NA, 5, 8, 1, NA)  
  
# All non-missing values of x  
x[!is.na(x)]  
#> [1] 10 3 5 8 1  
  
# All even (or missing!) values of x  
x[x %% 2 == 0]  
#> [1] 10 NA 8 NA
```

- If you have a named vector, you can subset it with a character vector:

```
x <- c(abc = 1, def = 2, xyz = 5)  
x[c("xyz", "def")]  
#> xyz def  
#> 5 2
```

Like with positive integers, you can also use a character vector to duplicate individual entries.

- The simplest type of subsetting is nothing, `x[]`, which returns the complete `x`. This is not useful for subsetting vectors, but it is useful when subsetting matrices (and other high-dimensional structures) because it lets you select all the rows or all the columns, by leaving that index blank. For example, if `x` is 2D,

`x[1, ]` selects the first row and all the columns, and `x[, -1]` selects all rows and all columns except the first.

To learn more about the applications of subsetting, read the “[Subsetting](#)” chapter of *Advanced R*.

There is an important variation of `[` called `[[`. `[[` only ever extracts a single element, and always drops names. It’s a good idea to use it whenever you want to make it clear that you’re extracting a single item, as in a for loop. The distinction between `[` and `[[` is most important for lists, as we’ll see shortly.

Exercises

1. What does `mean(is.na(x))` tell you about a vector `x`? What about `sum(!is.finite(x))`?
2. Carefully read the documentation of `is.vector()`. What does it actually test for? Why does `is.atomic()` not agree with the definition of atomic vectors above?
3. Compare and contrast `setNames()` with `purrr::set_names()`.
4. Create functions that take a vector as input and return:
 - a. The last value. Should you use `[` or `[[`?
 - b. The elements at even numbered positions.
 - c. Every element except the last value.
 - d. Only even numbers (and no missing values).
5. Why is `x[-which(x > 0)]` not the same as `x[x <= 0]`?
6. What happens when you subset with a positive integer that’s bigger than the length of the vector? What happens when you subset with a name that doesn’t exist?

Recursive Vectors (Lists)

Lists are a step up in complexity from atomic vectors, because lists can contain other lists. This makes them suitable for representing hierarchical or tree-like structures. You create a list with `list()`:

```
x <- list(1, 2, 3)
x
#> [[1]]
```

```
#> [1] 1
#>
#> [[2]]
#> [1] 2
#>
#> [[3]]
#> [1] 3
```

A very useful tool for working with lists is `str()` because it focuses on the *structure*, not the contents:

```
str(x)
#> List of 3
#> $ : num 1
#> $ : num 2
#> $ : num 3

x_named <- list(a = 1, b = 2, c = 3)
str(x_named)
#> List of 3
#> $ a: num 1
#> $ b: num 2
#> $ c: num 3
```

Unlike atomic vectors, `lists()` can contain a mix of objects:

```
y <- list("a", 1L, 1.5, TRUE)
str(y)
#> List of 4
#> $ : chr "a"
#> $ : int 1
#> $ : num 1.5
#> $ : logi TRUE
```

Lists can even contain other lists!

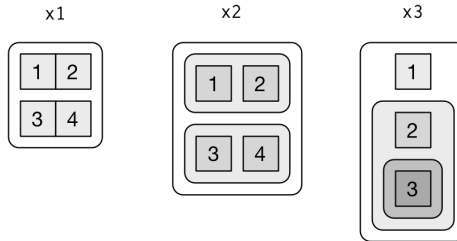
```
z <- list(list(1, 2), list(3, 4))
str(z)
#> List of 2
#> $ :List of 2
#> ..$ : num 1
#> ..$ : num 2
#> $ :List of 2
#> ..$ : num 3
#> ..$ : num 4
```

Visualizing Lists

To explain more complicated list manipulation functions, it's helpful to have a visual representation of lists. For example, take these three lists:

```
x1 <- list(c(1, 2), c(3, 4))
x2 <- list(list(1, 2), list(3, 4))
x3 <- list(1, list(2, list(3)))
```

I'll draw them as follows:



There are three principles:

- Lists have rounded corners. Atomic vectors have square corners.
- Children are drawn inside their parent, and have a slightly darker background to make it easier to see the hierarchy.
- The orientation of the children (i.e., rows or columns) isn't important, so I'll pick a row or column orientation to either save space or illustrate an important property in the example.

Subsetting

There are three ways to subset a list, which I'll illustrate with a:

```
a <- list(a = 1:3, b = "a string", c = pi, d = list(-1, -5))
```

- `[` extracts a sublist. The result will always be a list:

```
str(a[1:2])
#> List of 2
#> $ a: int [1:3] 1 2 3
#> $ b: chr "a string"
str(a[4])
#> List of 1
#> $ d:List of 2
#> ..$ : num -1
#> ..$ : num -5
```

Like with vectors, you can subset with a logical, integer, or character vector.

- `[[` extracts a single component from a list. It removes a level of hierarchy from the list:

```
str(y[[1]])
#> chr "a"
str(y[[4]])
#> logi TRUE
```

- `$` is a shorthand for extracting named elements of a list. It works similarly to `[[` except that you don't need to use quotes:

```
a$a
#> [1] 1 2 3
a[["a"]]
#> [1] 1 2 3
```

The distinction between `[` and `[[` is really important for lists, because `[[` drills down into the list while `[` returns a new, smaller list. Compare the preceding code and output with the visual representation in [Figure 16-2](#).

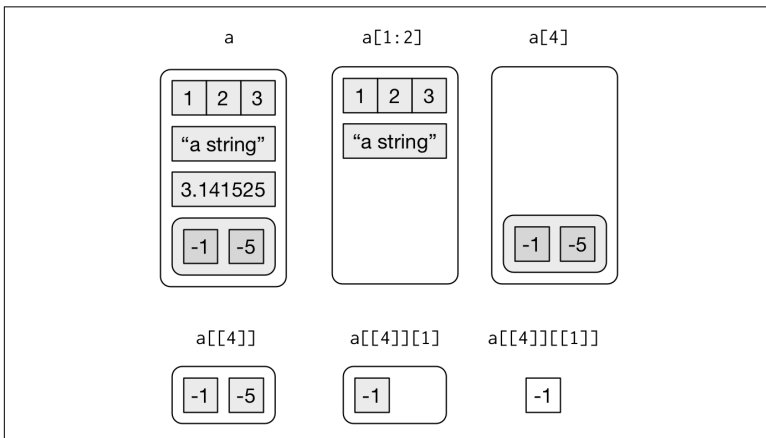


Figure 16-2. Subsetting a list, visually

Lists of Condiments

The difference between `[` and `[[` is very important, but it's easy to get confused. To help you remember, let me show you an unusual pepper shaker:



If this pepper shaker is your list `x`, then, `x[1]` is a pepper shaker containing a single pepper packet:



`x[2]` would look the same, but would contain the second packet.
`x[1:2]` would be a pepper shaker containing two pepper packets.

`x[[1]]` is:



If you wanted to get the content of the pepper package, you'd need `x[[1]][[1]]`:



Exercises

1. Draw the following lists as nested sets:
 - a. `list(a, b, list(c, d), list(e, f))`
 - b. `list(list(list(list(list(list(a))))))`
2. What happens if you subset a tibble as if you're subsetting a list?
What are the key differences between a list and a tibble?

Attributes

Any vector can contain arbitrary additional metadata through its *attributes*. You can think of attributes as a named list of vectors that can be attached to any object. You can get and set individual

attribute values with `attr()` or see them all at once with `attributes()`:

```
x <- 1:10
attr(x, "greeting")
#> NULL
attr(x, "greeting") <- "Hi!"
attr(x, "farewell") <- "Bye!"
attributes(x)
#> $greeting
#> [1] "Hi!"
#>
#> $farewell
#> [1] "Bye!"
```

There are three very important attributes that are used to implement fundamental parts of R:

- *Names* are used to name the elements of a vector.
- *Dimensions* (dims, for short) make a vector behave like a matrix or array.
- *Class* is used to implement the S3 object-oriented system.

You’ve seen names earlier, and we won’t cover dimensions because we don’t use matrices in this book. It remains to describe the class, which controls how *generic functions* work. Generic functions are key to object-oriented programming in R, because they make functions behave differently for different classes of input. A detailed discussion of object-oriented programming is beyond the scope of this book, but you can read more about it in *Advanced R*.

Here’s what a typical generic function looks like:

```
as.Date
#> function (x, ...)
#> UseMethod("as.Date")
#> <bytecode: 0x7fa61e0590d8>
#> <environment: namespace:base>
```

The call to “UseMethod” means that this is a generic function, and it will call a specific *method*, a function, based on the class of the first argument. (All methods are functions; not all functions are methods.) You can list all the methods for a generic with `methods()`:

```

methods("as.Date")
#> [1] as.Date.character as.Date.date      as.Date.dates
#> [4] as.Date.default as.Date.factor      as.Date.numeric
#> [7] as.Date.POSIXct   as.Date.POSIXlt
#> see '?methods' for accessing help and source code

```

For example, if `x` is a character vector, `as.Date()` will call `as.Date.character()`; if it's a factor, it'll call `as.Date.factor()`.

You can see the specific implementation of a method with `getS3method()`:

```

getS3method("as.Date", "default")
#> function (x, ...)
#> {
#>   if (inherits(x, "Date"))
#>     return(x)
#>   if (is.logical(x) && all(is.na(x)))
#>     return(structure(as.numeric(x), class = "Date"))
#>   stop(
#>     gettextf("do not know how to convert '%s' to class %s",
#>       deparse(substitute(x)), dQuote("Date")), domain = NA)
#> }
#> <bytecode: 0x7fa61dd47e78>
#> <environment: namespace:base>
getS3method("as.Date", "numeric")
#> function (x, origin, ...)
#> {
#>   if (missing(origin))
#>     stop("'origin' must be supplied")
#>   as.Date(origin, ...) + x
#> }
#> <bytecode: 0x7fa61dd463b8>
#> <environment: namespace:base>

```

The most important S3 generic is `print()`: it controls how the object is printed when you type its name at the console. Other important generics are the subsetting functions `[`, `[[`, and `$`.

Augmented Vectors

Atomic vectors and lists are the building blocks for other important vector types like factors and dates. I call these *augmented vectors*, because they are vectors with additional *attributes*, including *class*. Because augmented vectors have a class, they behave differently to the atomic vector on which they are built. In this book, we make use of four important augmented vectors:

- Factors
- Date-times and times
- Tibbles

These are described next.

Factors

Factors are designed to represent categorical data that can take a fixed set of possible values. Factors are built on top of integers, and have a `levels` attribute:

```
x <- factor(c("ab", "cd", "ab"), levels = c("ab", "cd", "ef"))
typeof(x)
#> [1] "integer"
attributes(x)
#> $levels
#> [1] "ab" "cd" "ef"
#>
#> $class
#> [1] "factor"
```

Dates and Date-Times

Dates in R are numeric vectors that represent the number of days since 1 January 1970:

```
x <- as.Date("1971-01-01")
unclass(x)
#> [1] 365

typeof(x)
#> [1] "double"
attributes(x)
#> $class
#> [1] "Date"
```

Date-times are numeric vectors with class `POSIXct` that represent the number of seconds since 1 January 1970. (In case you were wondering, “POSIXct” stands for “Portable Operating System Interface,” calendar time.)

```
x <- lubridate::ymd_hm("1970-01-01 01:00")
unclass(x)
#> [1] 3600
#> attr(,"tzzone")
#> [1] "UTC"
```

```
typeof(x)
#> [1] "double"
attributes(x)
#> $tzone
#> [1] "UTC"
#>
#> $class
#> [1] "POSIXct" "POSIXt"
```

The `tzone` attribute is optional. It controls how the time is printed, not what absolute time it refers to:

```
attr(x, "tzone") <- "US/Pacific"
x
#> [1] "1969-12-31 17:00:00 PST"

attr(x, "tzone") <- "US/Eastern"
x
#> [1] "1969-12-31 20:00:00 EST"
```

There is another type of date-times called `POSIXlt`. These are built on top of named lists:

```
y <- as.POSIXlt(x)
typeof(y)
#> [1] "list"
attributes(y)
#> $names
#> [1] "sec" "min" "hour" "mday" "mon" "year"
#> [7] "wday" "yday" "isdst" "zone" "gmtoff"
#>
#> $class
#> [1] "POSIXlt" "POSIXt"
#>
#> $tzone
#> [1] "US/Eastern" "EST" "EDT"
```

`POSIXlts` are rare inside the tidyverse. They do crop up in base R, because they are needed to extract specific components of a date, like the year or month. Since **lubridate** provides helpers for you to do this instead, you don't need them. `POSIXct`'s are always easier to work with, so if you find you have a `POSIXlt`, you should always convert it to a regular date-time with `lubridate::as_date_time()`.

Tibbles

Tibbles are augmented lists. They have three classes: `tbl_df`, `tbl`, and `data.frame`. They have two attributes: (column) names and `row.names`.

```

tb <- tibble::tibble(x = 1:5, y = 5:1)
typeof(tb)
#> [1] "list"
attributes(tb)
#> $names
#> [1] "x" "y"
#>
#> $class
#> [1] "tbl_df"      "tbl"        "data.frame"
#>
#> $row.names
#> [1] 1 2 3 4 5

```

Traditional `data.frames` have a very similar structure:

```

df <- data.frame(x = 1:5, y = 5:1)
typeof(df)
#> [1] "list"
attributes(df)
#> $names
#> [1] "x" "y"
#>
#> $row.names
#> [1] 1 2 3 4 5
#>
#> $class
#> [1] "data.frame"

```

The main difference is the class. The class of tibble includes “data.frame,” which means tibbles inherit the regular data frame behavior by default.

The difference between a tibble or a data frame and a list is that all of the elements of a tibble or data frame must be vectors with the same length. All functions that work with tibbles enforce this constraint.

Exercises

1. What does `hms::hms(3600)` return? How does it print? What primitive type is the augmented vector built on top of? What attributes does it use?
2. Try and make a tibble that has columns with different lengths. What happens?
3. Based on the previous definition, is it OK to have a list as a column of a tibble?

Iteration with purrr

Introduction

In [Chapter 15](#), we talked about how important it is to reduce duplication in your code by creating functions instead of copying and pasting. Reducing code duplication has three main benefits:

- It's easier to see the intent of your code, because your eyes are drawn to what's different, not what stays the same.
- It's easier to respond to changes in requirements. As your needs change, you only need to make changes in one place, rather than remembering to change every place that you copied and pasted the code.
- You're likely to have fewer bugs because each line of code is used in more places.

One tool for reducing duplication is functions, which reduce duplication by identifying repeated patterns of code and extracting them out into independent pieces that can be easily reused and updated. Another tool for reducing duplication is *iteration*, which helps you when you need to do the same thing to multiple inputs: repeating the same operation on different columns, or on different datasets. In this chapter you'll learn about two important iteration paradigms: imperative programming and functional programming. On the imperative side you have tools like `for` loops and `while` loops, which are a great place to start because they make iteration very explicit, so it's obvious what's happening. However, `for` loops are quite verbose,

and require quite a bit of bookkeeping code that is duplicated for every for loop. Functional programming (FP) offers tools to extract out this duplicated code, so each common for loop pattern gets its own function. Once you master the vocabulary of FP, you can solve many common iteration problems with less code, more ease, and fewer errors.

Prerequisites

Once you've mastered the for loops provided by base R, you'll learn some of the powerful programming tools provided by **purrr**, one of the tidyverse core packages.

```
library(tidyverse)
```

For Loops

Imagine we have this simple tibble:

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
```

We want to compute the median of each column. You *could* do it with copy-and-paste:

```
median(df$a)
#> [1] -0.246
median(df$b)
#> [1] -0.287
median(df$c)
#> [1] -0.0567
median(df$d)
#> [1] 0.144
```

But that breaks our rule of thumb: never copy and paste more than twice. Instead, we could use a for loop:

```
output <- vector("double", ncol(df)) # 1. output
for (i in seq_along(df)) {           # 2. sequence
  output[[i]] <- median(df[[i]])      # 3. body
}
output
#> [1] -0.2458 -0.2873 -0.0567 0.1443
```


Every for loop has three components:

output `output <- vector("double", length(x))`

Before you start the loop, you must always allocate sufficient space for the output. This is very important for efficiency: if you grow the for loop at each iteration using `c()` (for example), your for loop will be very slow.

A general way of creating an empty vector of given length is the `vector()` function. It has two arguments: the type of the vector (“logical,” “integer,” “double,” “character,” etc.) and the length of the vector.

sequence `i in seq_along(df)`

This determines what to loop over: each run of the for loop will assign `i` to a different value from `seq_along(df)`. It’s useful to think of `i` as a pronoun, like “it.”

You might not have seen `seq_along()` before. It’s a safe version of the familiar `1:length(l)`, with an important difference; if you have a zero-length vector, `seq_along()` does the right thing:

```
y <- vector("double", 0)
seq_along(y)
#> integer(0)
1:length(y)
#> [1] 1 0
```

You probably won’t create a zero-length vector deliberately, but it’s easy to create them accidentally. If you use `1:length(x)` instead of `seq_along(x)`, you’re likely to get a confusing error message.

body `output[[i]] <- median(df[[i]])`

This is the code that does the work. It’s run repeatedly, each time with a different value for `i`. The first iteration will run `output[[1]] <- median(df[[1]])`, the second will run `output[[2]] <- median(df[[2]])`, and so on.

That’s all there is to the for loop! Now is a good time to practice creating some basic (and not so basic) for loops using the following exercises. Then we’ll move on to some variations of the for loop that help you solve other problems that will crop up in practice.

Exercises

1. Write for loops to:
 - a. Compute the mean of every column in `mtcars`.
 - b. Determine the type of each column in `nycflights13::flights`.
 - c. Compute the number of unique values in each column of `iris`.
 - d. Generate 10 random normals for each of $\mu = -10, 0, 10$, and 100.

Think about the output, sequence, and body *before* you start writing the loop.

2. Eliminate the for loop in each of the following examples by taking advantage of an existing function that works with vectors:

```
out <- ""
for (x in letters) {
  out <- stringr::str_c(out, x)
}

x <- sample(100)
sd <- 0
for (i in seq_along(x)) {
  sd <- sd + (x[i] - mean(x)) ^ 2
}
sd <- sqrt(sd / (length(x) - 1))

x <- runif(100)
out <- vector("numeric", length(x))
out[1] <- x[1]
for (i in 2:length(x)) {
  out[i] <- out[i - 1] + x[i]
}
```

3. Combine your function writing and for loop skills:
 - a. Write a for loop that prints() the lyrics to the children's song "Alice the Camel."
 - b. Convert the nursery rhyme "Ten in the Bed" to a function. Generalize it to any number of people in any sleeping structure.

- c. Convert the song “99 Bottles of Beer on the Wall” to a function. Generalize to any number of any vessel containing any liquid on any surface.
4. It’s common to see for loops that don’t preallocate the output and instead increase the length of a vector at each step:

```
output <- vector("integer", 0)
for (i in seq_along(x)) {
  output <- c(output, lengths(x[[i]]))
}
output
```

How does this affect performance? Design and execute an experiment.

For Loop Variations

Once you have the basic for loop under your belt, there are some variations that you should be aware of. These variations are important regardless of how you do iteration, so don’t forget about them once you’ve mastered the FP techniques you’ll learn about in the next section.

There are four variations on the basic theme of the for loop:

- Modifying an existing object, instead of creating a new object.
- Looping over names or values, instead of indices.
- Handling outputs of unknown length.
- Handling sequences of unknown length.

Modifying an Existing Object

Sometimes you want to use a for loop to modify an existing object. For example, remember our challenge from [Chapter 15](#). We wanted to rescale every column in a data frame:

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
rescale01 <- function(x) {
```

```

  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}

df$a <- rescale01(df$a)
df$b <- rescale01(df$b)
df$c <- rescale01(df$c)
df$d <- rescale01(df$d)

```

To solve this with a for loop we again think about the three components:

Output

We already have the output—it's the same as the input!

Sequence

We can think about a data frame as a list of columns, so we can iterate over each column with `seq_along(df)`.

Body

Apply `rescale01()`.

This gives us:

```

for (i in seq_along(df)) {
  df[[i]] <- rescale01(df[[i]])
}

```

Typically you'll be modifying a list or data frame with this sort of loop, so remember to use `[[`, not `[`. You might have spotted that I used `[[` in all my for loops: I think it's better to use `[[` even for atomic vectors because it makes it clear that I want to work with a single element.

Looping Patterns

There are three basic ways to loop over a vector. So far I've shown you the most general: looping over the numeric indices with `for (i in seq_along(xs))`, and extracting the value with `x[[i]]`. There are two other forms:

- Loop over the elements: `for (x in xs)`. This is most useful if you only care about side effects, like plotting or saving a file, because it's difficult to save the output efficiently.
- Loop over the names: `for (nm in names(xs))`. This gives you a name, which you can use to access the value with `x[[nm]]`. This is useful if you want to use the name in a plot title or a filename.

If you're creating named output, make sure to name the results vector like so:

```
results <- vector("list", length(x))
names(results) <- names(x)
```

Iteration over the numeric indices is the most general form, because given the position you can extract both the name and the value:

```
for (i in seq_along(x)) {
  name <- names(x)[[i]]
  value <- x[[i]]
}
```

Unknown Output Length

Sometimes you might not know how long the output will be. For example, imagine you want to simulate some random vectors of random lengths. You might be tempted to solve this problem by progressively growing the vector:

```
means <- c(0, 1, 2)

output <- double()
for (i in seq_along(means)) {
  n <- sample(100, 1)
  output <- c(output, rnorm(n, means[[i]]))
}
str(output)
#>  num [1:202] 0.912 0.205 2.584 -0.789 0.588 ...
```

But this is not very efficient because in each iteration, R has to copy all the data from the previous iterations. In technical terms you get “quadratic” ($O(n^2)$) behavior, which means that a loop with three times as many elements would take nine (3^2) times as long to run.

A better solution is to save the results in a list, and then combine into a single vector after the loop is done:

```
out <- vector("list", length(means))
for (i in seq_along(means)) {
  n <- sample(100, 1)
  out[[i]] <- rnorm(n, means[[i]])
}
str(out)
#> List of 3
#> $ : num [1:83] 0.367 1.13 -0.941 0.218 1.415 ...
#> $ : num [1:21] -0.485 -0.425 2.937 1.688 1.324 ...
#> $ : num [1:40] 2.34 1.59 2.93 3.84 1.3 ...
```

```
str(unlist(out))
#>  num [1:144] 0.367 1.13 -0.941 0.218 1.415 ...
```

Here I've used `unlist()` to flatten a list of vectors into a single vector. A stricter option is to use `purrr::flatten_dbl()`—it will throw an error if the input isn't a list of doubles.

This pattern occurs in other places too:

- You might be generating a long string. Instead of `paste()`ing together each iteration with the previous, save the output in a character vector and then combine that vector into a single string with `paste(output, collapse = "")`.
- You might be generating a big data frame. Instead of sequentially `rbind()`ing in each iteration, save the output in a list, then use `dplyr::bind_rows(output)` to combine the output into a single data frame.

Watch out for this pattern. Whenever you see it, switch to a more complex result object, and then combine in one step at the end.

Unknown Sequence Length

Sometimes you don't even know how long the input sequence should be. This is common when doing simulations. For example, you might want to loop until you get three heads in a row. You can't do that sort of iteration with the `for` loop. Instead, you can use a `while` loop. A `while` loop is simpler than a `for` loop because it only has two components, a condition and a body:

```
while (condition) {
  # body
}
```

A `while` loop is also more general than a `for` loop, because you can rewrite any `for` loop as a `while` loop, but you can't rewrite every `while` loop as a `for` loop:

```
for (i in seq_along(x)) {
  # body
}

# Equivalent to
i <- 1
while (i <= length(x)) {
  # body
}
```

```
i <- i + 1
}
```

Here's how we could use a while loop to find how many tries it takes to get three heads in a row:

```
flip <- function() sample(c("T", "H"), 1)

flips <- 0
nheads <- 0

while (nheads < 3) {
  if (flip() == "H") {
    nheads <- nheads + 1
  } else {
    nheads <- 0
  }
  flips <- flips + 1
}
flips
#> [1] 3
```

I mention while loops only briefly, because I hardly ever use them. They're most often used for simulation, which is outside the scope of this book. However, it is good to know they exist so that you're prepared for problems where the number of iterations is not known in advance.

Exercises

1. Imagine you have a directory full of CSV files that you want to read in. You have their paths in a vector, `files <- dir("data/", pattern = "\\*.csv$", full.names = TRUE)`, and now want to read each one with `read_csv()`. Write the for loop that will load them into a single data frame.
2. What happens if you use `for (nm in names(x))` and `x` has no names? What if only some of the elements are named? What if the names are not unique?
3. Write a function that prints the mean of each numeric column in a data frame, along with its name. For example, `show_mean(iris)` would print:

```
show_mean(iris)
#> Sepal.Length: 5.84
#> Sepal.Width: 3.06
#> Petal.Length: 3.76
#> Petal.Width: 1.20
```

(Extra challenge: what function did I use to make sure that the numbers lined up nicely, even though the variable names had different lengths?)

4. What does this code do? How does it work?

```
trans <- list(
  disp = function(x) x * 0.0163871,
  am = function(x) {
    factor(x, labels = c("auto", "manual"))
  }
)
for (var in names(trans)) {
  mtcars[[var]] <- trans[[var]](mtcars[[var]])
}
```

For Loops Versus Functionals

For loops are not as important in R as they are in other languages because R is a functional programming language. This means that it's possible to wrap up for loops in a function, and call that function instead of using the for loop directly.

To see why this is important, consider (again) this simple data frame:

```
df <- tibble(
  a = rnorm(10),
  b = rnorm(10),
  c = rnorm(10),
  d = rnorm(10)
)
```

Imagine you want to compute the mean of every column. You could do that with a for loop:

```
output <- vector("double", length(df))
for (i in seq_along(df)) {
  output[[i]] <- mean(df[[i]])
}
output
#> [1]  0.2026 -0.2068  0.1275 -0.0917
```

You realize that you're going to want to compute the means of every column pretty frequently, so you extract it out into a function:

```
col_mean <- function(df) {
  output <- vector("double", length(df))
  for (i in seq_along(df)) {
```



```

    output[i] <- mean(df[[i]])
  }
  output
}

```

But then you think it'd also be helpful to be able to compute the median, and the standard deviation, so you copy and paste your `col_mean()` function and replace the `mean()` with `median()` and `sd()`:

```

col_median <- function(df) {
  output <- vector("double", length(df))
  for (i in seq_along(df)) {
    output[i] <- median(df[[i]])
  }
  output
}
col_sd <- function(df) {
  output <- vector("double", length(df))
  for (i in seq_along(df)) {
    output[i] <- sd(df[[i]])
  }
  output
}

```

Uh oh! You've copied and pasted this code twice, so it's time to think about how to generalize it. Notice that most of this code is for-loop boilerplate and it's hard to see the one thing (`mean()`, `median()`, `sd()`) that is different between the functions.

What would you do if you saw a set of functions like this?

```

f1 <- function(x) abs(x - mean(x)) ^ 1
f2 <- function(x) abs(x - mean(x)) ^ 2
f3 <- function(x) abs(x - mean(x)) ^ 3

```

Hopefully, you'd notice that there's a lot of duplication, and extract it out into an additional argument:

```

f <- function(x, i) abs(x - mean(x)) ^ i

```

You've reduced the chance of bugs (because you now have 1/3 less code), and made it easy to generalize to new situations.

We can do exactly the same thing with `col_mean()`, `col_median()`, and `col_sd()` by adding an argument that supplies the function to apply to each column:

```

col_summary <- function(df, fun) {
  out <- vector("double", length(df))
  for (i in seq_along(df)) {

```

```

      out[i] <- fun(df[[i]])
    }
    out
  }
  col_summary(df, median)
#> [1]  0.237 -0.218  0.254 -0.133
  col_summary(df, mean)
#> [1]  0.2026 -0.2068  0.1275 -0.0917

```

The idea of passing a function to another function is an extremely powerful idea, and it's one of the behaviors that makes R a functional programming language. It might take you a while to wrap your head around the idea, but it's worth the investment. In the rest of the chapter, you'll learn about and use the **purrr** package, which provides functions that eliminate the need for many common for loops. The apply family of functions in base R (`apply()`, `lapply()`, `tapply()`, etc.) solve a similar problem, but **purrr** is more consistent and thus is easier to learn.

The goal of using **purrr** functions instead of for loops is to allow you to break common list manipulation challenges into independent pieces:

- How can you solve the problem for a single element of the list? Once you've solved that problem, **purrr** takes care of generalizing your solution to every element in the list.
- If you're solving a complex problem, how can you break it down into bite-sized pieces that allow you to advance one small step toward a solution? With **purrr**, you get lots of small pieces that you can compose together with the pipe.

This structure makes it easier to solve new problems. It also makes it easier to understand your solutions to old problems when you re-read your old code.

Exercises

1. Read the documentation for `apply()`. In the second case, what two for loops does it generalize?
2. Adapt `col_summary()` so that it only applies to numeric columns. You might want to start with an `is_numeric()` function that returns a logical vector that has a `TRUE` corresponding to each numeric column.

The Map Functions

The pattern of looping over a vector, doing something to each element, and saving the results is so common that the **purrr** package provides a family of functions to do it for you. There is one function for each type of output:

- `map()` makes a list.
- `map_lgl()` makes a logical vector.
- `map_int()` makes an integer vector.
- `map_dbl()` makes a double vector.
- `map_chr()` makes a character vector.

Each function takes a vector as input, applies a function to each piece, and then returns a new vector that's the same length (and has the same names) as the input. The type of the vector is determined by the suffix to the map function.

Once you master these functions, you'll find it takes much less time to solve iteration problems. But you should never feel bad about using a for loop instead of a map function. The map functions are a step up a tower of abstraction, and it can take a long time to get your head around how they work. The important thing is that you solve the problem that you're working on, not write the most concise and elegant code (although that's definitely something you want to strive toward!).

Some people will tell you to avoid for loops because they are slow. They're wrong! (Well at least they're rather out of date, as for loops haven't been slow for many years). The chief benefit of using functions like `map()` is not speed, but clarity: they make your code easier to write and to read.

We can use these functions to perform the same computations as the last for loop. Those summary functions returned doubles, so we need to use `map_dbl()`:

```
map_dbl(df, mean)
#>      a      b      c      d
#> 0.2026 -0.2068 0.1275 -0.0917
map_dbl(df, median)
#>      a      b      c      d
#> 0.237 -0.218 0.254 -0.133
```

```
map_dbl(df, sd)
#>      a      b      c      d
#> 0.796 0.759 1.164 1.062
```

Compared to using a for loop, focus is on the operation being performed (i.e., `mean()`, `median()`, `sd()`), not the bookkeeping required to loop over every element and store the output. This is even more apparent if we use the pipe:

```
df %>% map_dbl(mean)
#>      a      b      c      d
#> 0.2026 -0.2068 0.1275 -0.0917
df %>% map_dbl(median)
#>      a      b      c      d
#> 0.237 -0.218 0.254 -0.133
df %>% map_dbl(sd)
#>      a      b      c      d
#> 0.796 0.759 1.164 1.062
```

There are a few differences between `map_*()` and `col_summary()`:

- All **purrr** functions are implemented in C. This makes them a little faster at the expense of readability.
- The second argument, `.f`, the function to apply, can be a formula, a character vector, or an integer vector. You'll learn about those handy shortcuts in the next section.
- `map_*()` uses ... ("**Dot-Dot-Dot (...)**" on page 284) to pass along additional arguments to `.f` each time it's called:

```
map_dbl(df, mean, trim = 0.5)
#>      a      b      c      d
#> 0.237 -0.218 0.254 -0.133
```

- The map functions also preserve names:

```
z <- list(x = 1:3, y = 4:5)
map_int(z, length)
#> x y
#> 3 2
```

Shortcuts

There are a few shortcuts that you can use with `.f` in order to save a little typing. Imagine you want to fit a linear model to each group in a dataset. The following toy example splits up the `mtcars` dataset into three pieces (one for each value of `cylinder`) and fits the same linear model to each piece:

```
models <- mtcars %>%
  split(.$cyl) %>%
  map(function(df) lm(mpg ~ wt, data = df))
```

The syntax for creating an anonymous function in R is quite verbose so **purrr** provides a convenient shortcut—a one-sided formula:

```
models <- mtcars %>%
  split(.$cyl) %>%
  map(~lm(mpg ~ wt, data = .))
```

Here I've used `.` as a pronoun: it refers to the current list element (in the same way that `i` referred to the current index in the `for` loop).

When you're looking at many models, you might want to extract a summary statistic like the R^2 . To do that we need to first run `summary()` and then extract the component called `r.squared`. We could do that using the shorthand for anonymous functions:

```
models %>%
  map(summary) %>%
  map_dbl(~.$r.squared)
#>      4      6      8
#> 0.509 0.465 0.423
```

But extracting named components is a common operation, so **purrr** provides an even shorter shortcut: you can use a string.

```
models %>%
  map(summary) %>%
  map_dbl("r.squared")
#>      4      6      8
#> 0.509 0.465 0.423
```

You can also use an integer to select elements by position:

```
x <- list(list(1, 2, 3), list(4, 5, 6), list(7, 8, 9))
x %>% map_dbl(2)
#> [1] 2 5 8
```

Base R

If you're familiar with the `apply` family of functions in base R, you might have noticed some similarities with the **purrr** functions:

- `lapply()` is basically identical to `map()`, except that `map()` is consistent with all the other functions in **purrr**, and you can use the shortcuts for `.` and `f`.

- Base `sapply()` is a wrapper around `lapply()` that automatically simplifies the output. This is useful for interactive work but is problematic in a function because you never know what sort of output you'll get:

```
x1 <- list(
  c(0.27, 0.37, 0.57, 0.91, 0.20),
  c(0.90, 0.94, 0.66, 0.63, 0.06),
  c(0.21, 0.18, 0.69, 0.38, 0.77)
)
x2 <- list(
  c(0.50, 0.72, 0.99, 0.38, 0.78),
  c(0.93, 0.21, 0.65, 0.13, 0.27),
  c(0.39, 0.01, 0.38, 0.87, 0.34)
)

threshold <- function(x, cutoff = 0.8) x[x > cutoff]
x1 %>% sapply(threshold) %>% str()
#> List of 3
#> $ : num 0.91
#> $ : num [1:2] 0.9 0.94
#> $ : num(0)
x2 %>% sapply(threshold) %>% str()
#> num [1:3] 0.99 0.93 0.87
```

- `vapply()` is a safe alternative to `sapply()` because you supply an additional argument that defines the type. The only problem with `vapply()` is that it's a lot of typing: `vapply(df, is.numeric, logical(1))` is equivalent to `map_lgl(df, is.numeric)`. One advantage of `vapply()` over **purrr**'s map functions is that it can also produce matrices—the map functions only ever produce vectors.

I focus on **purrr** functions here because they have more consistent names and arguments, helpful shortcuts, and in the future will provide easy parallelism and progress bars.

Exercises

1. Write code that uses one of the map functions to:
 - a. Compute the mean of every column in `mtcars`.
 - b. Determine the type of each column in `nycflights13::flights`.

- c. Compute the number of unique values in each column of `iris`.
 - d. Generate 10 random normals for each of $\mu = -10, 0, 10$, and 100.
2. How can you create a single vector that for each column in a data frame indicates whether or not it's a factor?
 3. What happens when you use the map functions on vectors that aren't lists? What does `map(1:5, runif)` do? Why?
 4. What does `map(-2:2, rnorm, n = 5)` do? Why? What does `map_dbl(-2:2, rnorm, n = 5)` do? Why?
 5. Rewrite `map(x, function(df) lm(mpg ~ wt, data = df))` to eliminate the anonymous function.

Dealing with Failure

When you use the map functions to repeat many operations, the chances are much higher that one of those operations will fail. When this happens, you'll get an error message, and no output. This is annoying: why does one failure prevent you from accessing all the other successes? How do you ensure that one bad apple doesn't ruin the whole barrel?

In this section you'll learn how to deal with this situation with a new function: `safely()`. `safely()` is an adverb: it takes a function (a verb) and returns a modified version. In this case, the modified function will never throw an error. Instead, it always returns a list with two elements:

`result`

The original result. If there was an error, this will be `NULL`.

`error`

An error object. If the operation was successful, this will be `NULL`.

(You might be familiar with the `try()` function in base R. It's similar, but because it sometimes returns the original result and it sometimes returns an error object it's more difficult to work with.)

Let's illustrate this with a simple example, `log()`:

```
safe_log <- safely(log)
str(safe_log(10))
#> List of 2
#> $ result: num 2.3
#> $ error : NULL
str(safe_log("a"))
#> List of 2
#> $ result: NULL
#> $ error :List of 2
#> ..$ message: chr "non-numeric argument to mathematical ..."
#> ..$ call : language .f(...)
#> ..- attr(*, "class")= chr [1:3] "simpleError" "error" ...
```

When the function succeeds the result element contains the result and the error element is `NULL`. When the function fails, the result element is `NULL` and the error element contains an error object.

`safely()` is designed to work with `map`:

```
x <- list(1, 10, "a")
y <- x %>% map(safely(log))
str(y)
#> List of 3
#> $ :List of 2
#> ..$ result: num 0
#> ..$ error : NULL
#> $ :List of 2
#> ..$ result: num 2.3
#> ..$ error : NULL
#> $ :List of 2
#> ..$ result: NULL
#> ..$ error :List of 2
#> .. ..$ message: chr "non-numeric argument to ..."
#> .. ..$ call : language .f(...)
#> .. ...- attr(*, "class")=chr [1:3] "simpleError" "error" ...
```

This would be easier to work with if we had two lists: one of all the errors and one of all the output. That's easy to get with `purrr::transpose()`:

```
y <- y %>% transpose()
str(y)
#> List of 2
#> $ result:List of 3
#> ..$ : num 0
#> ..$ : num 2.3
#> ..$ : NULL
#> $ error :List of 3
#> ..$ : NULL
```



```
#> ..$ : NULL
#> ..$ :List of 2
#> .. ..$ message: chr "non-numeric argument to ..."
#> .. ..$ call : language .f(...)
#> .. ..- attr(*, "class")=chr [1:3] "simpleError" "error" ...
```

It's up to you how to deal with the errors, but typically you'll either look at the values of *x* where *y* is an error, or work with the values of *y* that are OK:

```
is_ok <- y$error %>% map_lgl(is_null)
x[!is_ok]
#> [[1]]
#> [1] "a"
y$result[is_ok] %>% flatten_dbl()
#> [1] 0.0 2.3
```

purrr provides two other useful adverbs:

- Like `safely()`, `possibly()` always succeeds. It's simpler than `safely()`, because you give it a default value to return when there is an error:

```
x <- list(1, 10, "a")
x %>% map_dbl(possibly(log, NA_real_))
#> [1] 0.0 2.3 NA
```

- `quietly()` performs a similar role to `safely()`, but instead of capturing errors, it captures printed output, messages, and warnings:

```
x <- list(1, -1)
x %>% map(quietly(log)) %>% str()
#> List of 2
#> $ :List of 4
#> ..$ result : num 0
#> ..$ output : chr ""
#> ..$ warnings: chr(0)
#> ..$ messages: chr(0)
#> $ :List of 4
#> ..$ result : num NaN
#> ..$ output : chr ""
#> ..$ warnings: chr "NaNs produced"
#> ..$ messages: chr(0)
```

Mapping over Multiple Arguments

So far we've mapped along a single input. But often you have multiple related inputs that you need to iterate along in parallel. That's the job of the `map2()` and `pmap()` functions. For example, imagine you want to simulate some random normals with different means. You know how to do that with `map()`:

```
mu <- list(5, 10, -3)
mu %>%
  map(rnorm, n = 5) %>%
  str()
#> List of 3
#> $ : num [1:5] 5.45 5.5 5.78 6.51 3.18
#> $ : num [1:5] 10.79 9.03 10.89 10.76 10.65
#> $ : num [1:5] -3.54 -3.08 -5.01 -3.51 -2.9
```

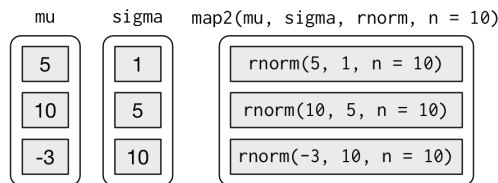
What if you also want to vary the standard deviation? One way to do that would be to iterate over the indices and index into vectors of means and sds:

```
sigma <- list(1, 5, 10)
seq_along(mu) %>%
  map(~rnorm(5, mu[[.]], sigma[[.]]) %>%
  str()
#> List of 3
#> $ : num [1:5] 4.94 2.57 4.37 4.12 5.29
#> $ : num [1:5] 11.72 5.32 11.46 10.24 12.22
#> $ : num [1:5] 3.68 -6.12 22.24 -7.2 10.37
```

But that obfuscates the intent of the code. Instead we could use `map2()`, which iterates over two vectors in parallel:

```
map2(mu, sigma, rnorm, n = 5) %>% str()
#> List of 3
#> $ : num [1:5] 4.78 5.59 4.93 4.3 4.47
#> $ : num [1:5] 10.85 10.57 6.02 8.82 15.93
#> $ : num [1:5] -1.12 7.39 -7.5 -10.09 -2.7
```

`map2()` generates this series of function calls:



Note that the arguments that vary for each call come *before* the function; arguments that are the same for every call come *after*.

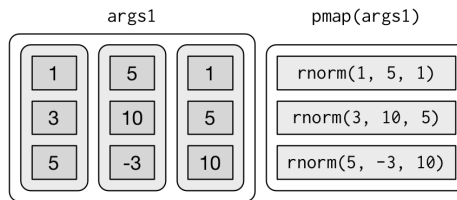
Like `map()`, `map2()` is just a wrapper around a for loop:

```
map2 <- function(x, y, f, ...) {  
  out <- vector("list", length(x))  
  for (i in seq_along(x)) {  
    out[[i]] <- f(x[[i]], y[[i]], ...)  
  }  
  out  
}
```

You could also imagine `map3()`, `map4()`, `map5()`, `map6()`, etc., but that would get tedious quickly. Instead, **purrr** provides `pmap()`, which takes a list of arguments. You might use that if you wanted to vary the mean, standard deviation, and number of samples:

```
n <- list(1, 3, 5)  
args1 <- list(n, mu, sigma)  
args1 %>%  
  pmap(rnorm) %>%  
  str()  
#> List of 3  
#> $ : num 4.55  
#> $ : num [1:3] 13.4 18.8 13.2  
#> $ : num [1:5] 0.685 10.801 -11.671 21.363 -2.562
```

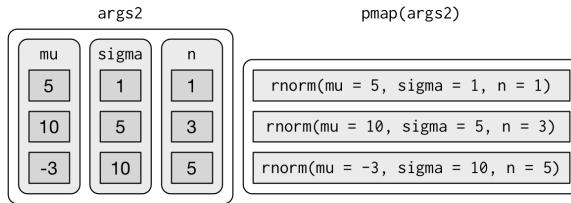
That looks like:



If you don't name the elements of list, `pmap()` will use positional matching when calling the function. That's a little fragile, and makes the code harder to read, so it's better to name the arguments:

```
args2 <- list(mean = mu, sd = sigma, n = n)  
args2 %>%  
  pmap(rnorm) %>%  
  str()
```

That generates longer, but safer, calls:



Since the arguments are all the same length, it makes sense to store them in a data frame:

```
params <- tribble(
  ~mean, ~sd, ~n,
  5,      1,  1,
  10,     5,  3,
  -3,    10,  5
)
params %>%
  pmap(rnorm)
#> [[1]]
#> [1] 4.68
#>
#> [[2]]
#> [1] 23.44 12.85 7.28
#>
#> [[3]]
#> [1] -5.34 -17.66 0.92 6.06 9.02
```

As soon as your code gets complicated, I think a data frame is a good approach because it ensures that each column has a name and is the same length as all the other columns.

Invoking Different Functions

There's one more step up in complexity—as well as varying the arguments to the function you might also vary the function itself:

```
f <- c("runif", "rnorm", "rpois")
param <- list(
  list(min = -1, max = 1),
  list(sd = 5),
  list(lambda = 10)
)
```